

DAY 4 · CLAUDE SKILLS, SUBAGENTS & AUTOMATION

ONE SKILL, ONE TOOL

Build a real Claude Skill for one task, and connect it to a real tool.

THE CONSIGNA

“Pick one task you would genuinely do more than once. **Narrow beats impressive.** ... Connect at least one MCP so Claude can act on a real tool, not just talk about it.”

A description that fires, the tools it may touch, the steps in order, the standard, and the exact output shape — then run it twice with different inputs.

WHAT I BUILT

SKILL	alm-codebase-lookup
THE ONE TASK	Answer a question about an ALM product from its code graph, instead of reading the repo.
MCP	codebase-memory-mcp
WHY THIS TASK	I have a dozen Python products of 200 to 3,000 lines each. Before touching one I re-read it, and the answer is always two functions deep.

WHY IT IS NOT A TOY

Both runs were against real, shipped products. Neither question was about the code I wrote for this homework.

Two real defects came out of it: a **silent failure path** in shipped code, and a duplicated module tree that will mislead any future query. Neither was the question asked.

DELIVERABLE · 1 OF 3

THE SKILL.MD

Name, description, tools, steps, standard, output shape.

NAME & DESCRIPTION

The description is the line Claude reads to decide whether this Skill fits the moment, so it names **triggers**, not topics.

Fires on “what does X do in <product>”, “where is X handled”, “how does <product> do Y”, “before I touch <product>”. Not for the bootcamp workspace, not for a repo that was never indexed, and not for writing or refactoring code.

The negative half matters as much as the positive: a skill that fires on everything fires correctly on nothing.

TOOLS — NARROWED DELIBERATELY

ALLOWED codebase-memory-mcp — eight tools, listed by name

EXCLUDED Read · Grep · Glob · Bash

OTHER MCPS ElevenLabs, Composio and claude-mem were available and left out.

The exclusion is the design. Reading a file is what this skill exists to avoid, so it is allowed only as a **declared** fallback — never silently.

THE ONE HARD RULE

The graph **contains credentials**. `generar_licencia.py` and the licensing code define a `SECRET_KEY`, and those files are indexed — so a plausible question resolves to a function holding a live key.

The danger is not a tool reaching for a credentials file. It is a **reasonable question whose answer happens to be a secret**. The `file:line` is the answer; the value never is.

DELIVERABLE · 2 OF 3

THE STEPS & OUTPUT

Plain English, in order, the way you would explain it to a new hire.

SIX STEPS

- 1 **Resolve.** Read the project name from list_projects — never construct it.
- 2 **Ensure indexed.** index_status first; index only if absent.
- 3 **Find.** search_graph with the question's own words; Cypher when that misses.
- 4 **Read the body** — the function, never the file.
- 5 **Follow the edges** for flow or impact: trace_path.
- 6 **Fallback** — allowed, but it must be declared in the output.

THE EXACT OUTPUT SHAPE

```
PRODUCT      comparador-precios
QUESTION     <the question, restated in one line>

ANSWER       <at most four sentences>

EVIDENCE
  comparador.py:54  buscar_carrefour() – builds the URL, parses the JSON
  comparador.py:12  SUPERMERCADOS – the registry of configured stores

SOURCE       graph | FALLBACK: <file> – <why the graph was not enough>
UNVERIFIED   <what the graph did not contain – or: none>
```

UNVERIFIED is never omitted. If the graph answered everything, it says none. A field you are forced to fill is a field you cannot quietly skip.

IDEMPOTENT BY CONSTRUCTION

Step 2 is why the skill is safe to run twice. On the second run the repo is already indexed, nothing is rebuilt, and the answer is faster — designed for reruns from day one, not patched afterwards.

DELIVERABLE · 3 OF 3

THE STANDARD

What counts as done well, not just done.

FOUR BARS

- 1 Every claim carries a file:line. A sentence that asserts behaviour without one is a guess wearing an answer's clothes.
- 2 The answer was built **without reading a whole file**. If one was read, SOURCE says FALLBACK and names it.
- 3 What the graph does not know is **stated, not papered over**.
- 4 **Four sentences, maximum**. Needing more means the question was too broad — say so, and ask which part to answer first.

Bar 4 is the one people argue with. It is also the one that keeps this skill from turning into *"read me the whole product."*

BUILT AGAINST THE FIVE PITFALLS

The class lists five ways automation fails. Each one is answered somewhere concrete in the file — that is why the file has the shape it has.

PITFALL	ANSWERED BY
No error handling	Step 3 → step 6. A miss becomes UNVERIFIED, never a guess.
No idempotency	index_status before index_repository.
No human checkpoint	N/A — the skill is read-only and cannot change a product. Say so rather than inventing a gate.
Scope creep on tools	One MCP, eight tools, four exclusions.
No test run	Two runs, on products whose answers I could check.

THE TECHNIQUE THAT CAME FROM THE ROOM

The instructor added a step the slide does not carry: *"I want you to analyze everything that can go wrong with this skill."* The table above is that pass, done. It is what turns a stated quality bar into one you could argue with.

EVIDENCE · RUN IT AT LEAST TWICE

TWO RUNS, TWO PRODUCTS

Different products, different questions. Neither is a fixture.

RUN 1 — COMPARADOR DE PRECIOS

If one supermarket fails to respond, does the whole comparison break, or does it degrade?

COMPARADOR_GUI.PY:216 · :698 — COMPARADOR.PY:54, :101

Answer, condensed. It degrades — but silently. A dead store returns an empty list instead of an error, and the caller never checks whether a store returned anything. The user sees a shorter comparison with no indication a store is missing.

That is pitfall 01 from the class, found in shipped code: *"it breaks silently and the client finds it first."*

SOURCE: graph · UNVERIFIED: whether the UI signals a partial result downstream.

RUN 2 — COPILOTO HAKIM

How does the agenda decide when to fire a notification?

AGENDA.PY:365, :429 — APP.PY:1419

Answer, condensed. A polling watchdog, not a scheduler. Four conditions in one SQL pass — not done, expired, under the notice cap, past the re-notify cooldown. Idempotent on purpose: the UPDATE repeats the expiry condition in its WHERE, so two simultaneous passes cannot notify twice.

SOURCE: graph · UNVERIFIED: the values of the three config constants.

WHAT THE TWO RUNS PROVE

Answers came from the graph	both runs
Every claim carries a file:line	ten citations
States what it could not check	both runs
Files opened	none

DAY 4 · TRANSCRIPT VS. SLIDES

SPOKEN VS. WRITTEN

The task page is eight lines. The constraints that bind it are not on it.

WHAT WAS SAID THAT THE SLIDE DOES NOT CARRY

- 1 *"Analyze everything that can go wrong with this skill."* — not on the slide. It is the technique for hitting the stated quality bar.
- 2 The upload takes **folders and zips**, not only a lone SKILL.md.
- 3 Skills are **private by default**; sharing is deliberate. The share channel is the zip.
- 4 Code execution is **required** for skills — without it they do not run at all.

And the reverse: **"run it at least twice"** was *not* said aloud. It is on the slide. The slide wins — two runs.

THE DEMO CONTRADICTED ITS OWN RULE

The instructor stated the rule flatly: *"whenever you're posting to LinkedIn, I wouldn't automate it. I will want to check the posts before they run."* Minutes earlier, in the same live build, he answered **"schedule automatically"** — and then had to go kill the routine.

The class's third pitfall, committed by the person teaching it. Worth keeping: the rule is easy to state and hard to execute, which is exactly why it belongs in a file rather than in memory.

WHERE THIS LEAVES THE DELIVERABLE

READY	Skill built, installed, run twice, MCP connected and scoped, evidence recorded, zip packaged.
RECORDED	The walkthrough below — what the skill does, in about three minutes.
PENDING	The upload to Claude, and the submission.

THE WALKTHROUGH

LOOM · 3 MIN

<https://www.loom.com/share/bf55bb4a049d4129b4c27c8007c09402>

What the skill actually does, walked through once — the task it replaces, the one tool it touches, and the two defects it found on its first two runs.